CSCI 3081W -- The Arena -- Code Review Preparation

Iteration 2

Architecture Deep Dive

Strategy Pattern . Adapter Pattern . Composite Pattern . Decorator Pattern

This document is sourced directly from the codebase & README2.md to ensure accuracy.

1 The Iteration 1 -> 2 Conceptual Shift

In Iteration 1, motions were tightly coupled to specific entity classes. A robot knew how to handle keyboard input; a light knew how to move back-and-forth. The logic lived inside each entity's Update() method and was not reusable. Iteration 2 represents a fundamental architectural refactor: motion is extracted into a separate class hierarchy (the Strategy Pattern), entities hold a pointer to an IMotion strategy, and MotionFactory constructs the correct IMotion from JSON at load time. Any entity can now use any motion -- a predator can bounce, a water entity can orbit, a robot can follow a composite Braitenberg tree.

Aspect	Iteration 1 (Before)	Iteration 2 (After)
Motion Location	Inside entity Update()	Separate IMotion class hierarchy
Who can move?	Only the entity it was coded into	ANY entity via IMotion* pointer
Motion selection	Hard-coded if/else in Update()	MotionFactory reads JSON at runtime
Combining motions	Not possible	Composite pattern: motion tree
Sensor-driven BV	Light-only, hard-wired	Any entity type via EntitySensor
Predator entity	Did not exist	Decorator wrapping, disguise logic
Energy / Water	Did not exist	Full charge drain & recharge system
Design patterns	None formally	Strategy, Adapter, Composite, Decorator, Template Method

KEY INSIGHT -- Why the Strategy Pattern?

Entities now store an IMotion* motion pointer (see EntityBase.h line 134).
Robot::Update() calls motion->Move(*this, dt) -- it doesn't care WHAT motion is used.
This makes adding new motions trivial: implement IMotion, register in MotionFactory, done.
It also enables the Composite pattern: a motion can contain OTHER motions as children.

2 Simple Motions -- IMotion Interface & Implementations

All motion classes live under include/sim/motion/ and src/sim/motion/. Every motion implements the IMotion interface -- a single pure-virtual method Move(IEntity&, double dt). This is the Strategy Pattern contract.

```
// include/sim/motion/IMotion.h
class IMotion {
 public:
  virtual ~IMotion() {}
  virtual void Move(IEntity& entity, double dt) = 0;      // THE contract
  virtual void HandleKeyEvent(const string& key, bool pressed) {}  // optional
};
```

Each concrete motion dynamic_cast<EntityBase*>(&entity) to get position/direction setters. If the cast fails the motion silently returns -- this makes it safe to assign any motion to any entity.

2.1 Default Motion

Does nothing. Used when an entity should be stationary. JSON: {"type": "default"}. This replaces the Iteration 1 pattern of simply not calling a move method.

2.2 Bounce Motion -- BounceMotion.cpp

Moves the entity forward at a constant speed, bouncing off arena walls. On first call it reads the entity's initial direction and computes velocity = dir * speed. On every wall collision it applies the reflection formula:

```
// v' = v - 2*(dot(n,v))*n    (n = wall outward normal)
// Right wall: n = (-1,0,0)    Left wall: n = (1,0,0)
// Bottom:     n = (0,-1,0)    Top:       n = (0,1,0)
double dot = vel[0]*n[0] + vel[1]*n[1] + vel[2]*n[2];
vel = vel - n * (2.0 * dot);
// Then clamp position so entity stays inside arena.
```

NOTE: In Iteration 1 the bounce was implemented inside LightObject::Update() using booleans to flip a direction. Now it is a proper physics reflection that works on any entity.

2.3 Keyboard Motion -- KeyboardMotion.cpp

Listens for HandleKeyEvent() calls from SimulationModel and maintains a std::set<string> keysPressed. On Move() it sums the unit direction vectors for all currently held keys, normalises the result (enabling true diagonal movement at 0.707 per axis), clamps to arena bounds, then calls entity.SetPosition() / SetDirection() / SetSpeed(). Previously only the Robot class received key events; now any entity assigned KeyboardMotion responds.

2.4 Back-and-Forth Motion -- BackAndForthMotion.cpp

Oscillates between P1 and P2 using a parametric t value (0.0 - 1.0 along the segment). Each tick advances t by (speed*dt)/segmentLength, flips direction at t=0 and t=1. Position is computed as P = A + t*(B-A), which avoids floating-point drift accumulation over many frames -- a correctness improvement over Iteration 1.

2.5 Around Motion -- AroundMotion.cpp

Calculates an orbit around a fixed centre point. On the first call it reads the entity's current position to determine the initial angle (atan2(dy,dx)), so placement in the JSON controls the starting point on the orbit. Each tick: angle -= speed*dt/radius. Positive speed = clockwise; negative = counter-clockwise. Direction is set to the tangent (-sin(angle), cos(angle), 0) so the entity always faces forward.

3 Braitenberg & Differential Drive -- Adapter Pattern

Iteration 1's Braitenberg Vehicle was a monolithic entity class with the sensor-to-wheel logic embedded directly. Iteration 2 extracts this into two layers using the Adapter Pattern:

* **DDMotionBase (Adapter Base)** Wraps DifferentialDriveKinematics into an IMotion. Holds leftWheelVelocity, rightWheelVelocity, maxSpeed, and the kinematics object. Sub-classes write to those wheel variables; DDMotionBase converts them to entity position/direction.
* **Braitenberg (Abstract Adapter)** Extends DDMotionBase. Adds left/right EntitySensor pointers and arena bounds. Provides ifOutsideReverse() helper that flips velocity when an entity strays past the arena boundary.
* **Concrete behaviors (Love / Fear / Aggression / Explore)** Each is a final class extending Braitenberg. Each overrides update() to read sensor values and set wheel velocities according to the Braitenberg wiring table.

```
// include/sim/motions/DifferentialDriveMotion.h
class DDMotionBase : public IMotion {
  protected:
    double leftWheelVelocity;   // set by sub-class
    double rightWheelVelocity;
    double maxSpeed;
    double radius;
    DifferentialDriveKinematics kinematics;
};

// include/sim/motions/braitenberg/Braitenberg.h
class Braitenberg : public DDMotionBase {
  protected:
    EntitySensor* leftSensor;
    EntitySensor* rightSensor;
    double arenaWidth, arenaHeight;
    void ifOutsideReverse(Vector3& vel, Vector3& pos, Vector3& oldPose);
};
```

3.1 EntitySensor -- Generic Target Detection

EntitySensor is the key to Iteration 2's flexibility. In Iteration 1, the BV's sensors always looked for Light objects. Now EntitySensor takes a const std::type_info& targetType at construction, so the same sensor code can detect any entity type at runtime.

```
// include/sim/entities/EntitySensor.h  (key excerpt)
class EntitySensor {
 public:
  EntitySensor(int angleOffset, double radiusOffset,
               const SimulationModel& model,
               const std::type_info& targetType);  // <-- runtime type!

  void UpdateLocation(Vector3 robotLoc, Vector3 robotDir);
  double getReading();      // returns 1800 / 1.08^distance

 private:
  double FindNearestEntityDistance();   // scans all entities, matches typeid
};
```

Sensor reading formula: reading = max(1800 / 1.08^distance, 0.0001). This gives an exponentially decaying signal --

strong when close, near-zero when far. UpdateLocation() rotates the sensor position by angleOffset degrees around the entity's heading using a 2D rotation matrix, placing left (+40°) and right (-40°) sensors.

3.2 Braitenberg Behaviors -- Wiring Table

The four behaviors each wire sensor readings to wheel velocities differently:

Behavior	Left Wheel	Right Wheel	Effect
Love	leftSensor	rightSensor	Follows target slowly, stays close
Fear	rightSensor	leftSensor	Turns away and accelerates from target
Aggression	leftSensor	rightSensor	Charges directly at target at max speed
Explore	rightSensor	leftSensor	Wanders toward weaker signal areas

KEY INSIGHT -- Why Adapter Pattern?

DifferentialDriveKinematics is existing physics code. It needs wheel velocities as input.
Braitenberg behaviors produce wheel velocities from sensor readings.
DDMotionBase ADAPTS the kinematics class to the IMotion interface -- classic Adapter.
The simulation only calls motion->Move(); it never knows kinematics exist.
Sub-classes (Love/Fear/etc.) only set leftWheelVelocity & rightWheelVelocity -- no physics.

4 Composite Motion Tree -- Composite & Template Method Patterns

Because IMotion is an interface AND CompositeMotion also implements IMotion, any motion can contain other motions as children. This is the Composite Pattern: a tree where internal nodes are container motions and leaves are simple or Braitenberg motions. MotionFactory builds this tree recursively from nested JSON.

4.1 CompositeMotion -- Sequential Execution

The simplest container. Stores a vector<IMotion*> children. On Move(), iterates over every child and calls child->Move(entity, dt) in order. All children modify the same entity sequentially -- later children can override what earlier children set. Typical use: first child handles wall avoidance (Bounce), second child handles steering (Braitenberg). HandleKeyEvent() propagates to all children so keyboard input reaches any nested KeyboardMotion.

```
// src/sim/motion/CompositeMotion.cpp
void CompositeMotion::Move(IEntity& entity, double dt) {
  for (IMotion* child : children) {
    child->Move(entity, dt);   // sequential, in order
  }
}
```

4.2 ClosestSwitchMotion -- Context-Aware Switching

Takes a list of entity-types and a matching list of child motions. On every Move() call it scans ALL simulation entities to find the single nearest entity whose type matches one of the registered types. The INDEX of that type determines which child motion executes. Result: a robot can have completely different strategies depending on what is nearest -- explore light, love robots, fear predators, all in one motion.

```
// src/sim/motion/ClosestSwitchMotion.cpp  (simplified)
void ClosestSwitchMotion::Move(IEntity& entity, double dt) {
  int closestTypeIdx = -1;
  double closestDist  = DOUBLE_MAX;
  for (const IEntity* other : model.GetEntities()) {
    for (int i = 0; i < entityTypes.size(); ++i) {
      if (other->GetTypeName() == entityTypes[i]) {
        dist = euclidean(entity, other);
        if (dist < closestDist) { closestDist = dist; closestTypeIdx = i; }
      }
    }
  }
  children[closestTypeIdx]->Move(entity, dt);  // activate ONE child
}
```

4.3 WeightedMotion -- Template Method Pattern (Abstract)

Abstract base class. Move() calls the pure-virtual GetWeights() to get a weight per child, normalises them so they sum to 1.0, then runs each child motion on a COPY of the entity's current position/direction. The resulting positions and directions are blended using a weighted average. This is the Template Method Pattern: the algorithm skeleton (normalize, run, blend) is in WeightedMotion; the policy (HOW to compute weights) is in sub-classes.

```
// src/sim/motion/WeightedMotion.cpp  (core algorithm)
void WeightedMotion::Move(IEntity& entity, double dt) {
  vector<double> weights = GetWeights(entity, dt);  // template method hook
  double total = sum(weights);
  Vector3 blendedPos(0,0,0), blendedDir(0,0,0);
  for (int i = 0; i < children.size(); ++i) {
```

```
    double w = weights[i] / total;
    e->SetPosition(originalPos); e->SetDirection(originalDir);  // reset
    children[i]->Move(entity, dt);                               // run child
    blendedPos += w * e->GetPositionVector();                    // accumulate
    blendedDir += w * e->GetDirectionVector();
  }
  e->SetPosition(blendedPos);
  e->SetDirection(blendedDir.Normalized());
}
virtual vector<double> GetWeights(IEntity&, double) = 0;  // sub-class fills
```

4.4 PercentMotion -- Fixed Weights

Concrete WeightedMotion. Weights are fixed percentages supplied in the JSON (e.g. [0.3, 0.7]). GetWeights() just returns those stored values. Use case: always 30% bounce, 70% Braitenberg-explore.

4.5 ChargeLevelMotion -- Energy-Adaptive Weights

Concrete WeightedMotion. Stores a list of energy thresholds. GetWeights() calls entity.GetCharge() via an EntityBase dynamic_cast and returns weights that shift as the robot's charge decreases -- e.g. when charge is high the robot explores freely; when charge drops below a threshold it prioritises seeking energy sources. This creates emergent survival behaviour.

4.6 InverseWeightedMotion -- Proximity-Driven Weights

Concrete WeightedMotion. For each registered entity-type, scans all entities within search-radius to find the nearest one, then computes W_i = 1/distance. GetWeights() returns the raw inverse-distance weights; WeightedMotion normalises them. Result: the motion with the highest weight is the one whose target type is CLOSEST. A water blob 20 units away overpowers a light 200 units away.

```
// src/sim/motion/InverseWeightedMotion.cpp
for (int i = 0; i < entityTypes.size(); ++i) {
  double nearestDist = DOUBLE_MAX;
  for (const IEntity* other : model.GetEntities()) {
    if (other->GetTypeName() == entityTypes[i]) {
      dist = euclidean(entity.pos, other->pos);
      if (dist < searchRadius && dist < nearestDist) nearestDist = dist;
    }
  }
  result[i] = (nearestDist > 0.0001) ? 1.0/nearestDist : 1e6;  // W_i = 1/d
}
// WeightedMotion then normalises: P_i = W_i / sum(W)
```

Composite Tree -- Real JSON Example from README2.md

```
{"type": "inverse-weighted", "entity-type": ["water","energy","light"], "search-radius": 500,
 "children": [
  {"type": "braitenberg_behavior", "behavior": "fear",       "target": "water"},
  {"type": "braitenberg_behavior", "behavior": "aggression", "target": "energy"},
  {"type": "braitenberg_behavior", "behavior": "explore",    "target": "light"}
 ]
}
```

Result: if water is closest -> mostly Fear-Water; if energy is closest -> mostly Aggression-Energy.

5 Entity Ecosystem -- Robot, BV, Predator, Water

5.1 EntityBase -- The New Base Class

In Iteration 2 all simulation entities extend EntityBase (not IEntity directly). EntityBase provides: pos, dir, speed, radius, charge, motion* -- the common state. It exposes SetMotion(IMotion*), GetCharge(), SetCharge(). The GetType() / GetTypeName() system uses typeid(T) stored at construction time, enabling O(1) type checks without dynamic_cast in hot paths (EntitySensor uses this).

5.2 Robot -- Energy-Constrained Mover

Robots received two major changes: (1) they now accept an IMotion* instead of hard-coded logic, and (2) they drain their charge every tick.

```
// src/sim/entities/Robot.cpp  -- Update()
void Robot::Update(double dt) {
  Vector3 prevPos = pos;
  motion->Move(*this, dt);                      // delegate to strategy

  double dx = pos[0] - prevPos[0];
  double dy = pos[1] - prevPos[1];
  double horizontal_distance = sqrt(dx*dx + dy*dy);
  // Formula from spec (terrain flat in Iter 2, so vertical terms = 0):
  charge -= (horizontal_distance + 0.01) * (dt/50.0) * (radius/50.0);
  if (charge < 0.0) charge = 0.0;
}
```

The formula means: faster movement, larger robots, and longer time steps all drain more charge. When terrain is non-flat (Iteration 3), positive_vertical_distance is multiplied by 10 and negative by 0.1.

5.3 BraitenbergVehicle -- Sensor-Driven Robot

Extends EntityBase (NOT Robot, to avoid multiple inheritance issues). Constructor takes left/right EntitySensor pointers and an IMotion*. In Update() it first calls UpdateLocation() on both sensors so they track the BV's current heading, then calls motion->Move(). Energy drain applies the constant idle term only (0.01 * dt/50 * radius/50) since the BV's actual motion displacement is harder to track separately from the sensor motion system.

```
// src/sim/entities/BraitenbergVehicle.cpp
void BraitenbergVehicle::Update(double dt) {
  rightSensor->UpdateLocation(pos, dir);  // reposition sensors first
  leftSensor->UpdateLocation(pos, dir);
  if (motion) motion->Move(*this, dt);    // strategy runs with fresh sensor data
  charge -= 0.01 * (dt/50.0) * (radius/50.0);
  if (charge < 0.0) charge = 0.0;
}
```

5.4 Predator Entity -- Decorator Pattern

Predators are defined in the spec as using the DECORATOR PATTERN -- they wrap themselves around another entity's visual/type identity to disguise themselves. Key rules:

* **Disguise Mode** Predator JSON includes a 'disguise' field specifying which entity type to mimic (e.g. {"type": "light"}). The predator appears as that type to sensors.
* **Attack Trigger** When a Predator using Braitenberg AGGRESSION motion intersects a Robot/BV, it immediately drains ALL the target's energy (charge = 0) and uses it to recharge itself.
* **Reveal on Strike** The moment it attacks, it must reveal itself as type 'Predator'. Only aggressive Braitenberg motion can

trigger an attack.

* **Energy Rules** Predators follow the same energy drain formula as Robots and must recharge at Energy entities.

```
// Predator JSON declaration:
{
  "type": "predator",
  "disguise": {"type": "light"},    // appears as Light to EntitySensor
  "motion": {"type": "bounce", "speed": 100.0},
  "capacity": 100.0
}
// When predator uses braitenberg_behavior aggression + intersects Robot:
//   robot->SetCharge(0.0);
//   predator->SetCharge(predator->GetCharge() + stolenCharge);
//   predator->RevealType();   // switches typeid back to Predator
```

5.5 Water Entity -- Environmental Hazard

Water is a new entity type that damages passing Robots. If a Robot's bounding sphere intersects with a Water entity's bounding sphere, the robot charge drains at a constant high rate each tick:

```
// Drain formula when robot intersects water:
robot_charge -= capacity * 0.1 * dt;
// At dt=1 this removes 10% of max capacity per second -- very aggressive.
```

6 Light, Energy & Water -- Environmental Interaction Layer

6.1 Light Entity -- Sensor Intensity Source

Light unchanged in structure but gains new significance: sensors use intensity as a multiplier. Intensity = 100.0 / radius. A smaller light is MORE intense (higher sensor signal per unit distance).

Light.cpp still uses the Iteration-1-era internal motion system (back_and_forth / around / default) because Light predates the IMotion refactor. This is a deliberate carry-over noted in SimulationModel: when type == 'light', it passes the raw JSON motion object directly to the Light constructor rather than routing through MotionFactory.

```
// src/sim/SimulationModel.cpp
if (type == "light") {
  // Light uses OLD internal motion system (sim/motions/) not MotionFactory
  return new Light(initPos, initDir, radius, data["motion"]);
}
```

6.2 Energy Entity -- Light Sub-class with Recharge

Energy is a specialisation of Light (electromagnetic radiation). Two distinguishing behaviours:

* **Sensor Masking** EntitySensor CANNOT identify an Energy entity as 'energy' until the sensor is within 5 * radius of the energy source. Outside that range it appears as generic Light.

* **Recharge Formula** When a Robot's position intersects an Energy entity, per tick: charge += time * intensity / 5.0. At full intensity this takes 5 seconds to fully recharge an empty robot.

```
// Recharge on intersection:
double intensity = 100.0 / energy.GetRadius();
robot.charge += dt * intensity / 5.0;
if (robot.charge > robot.capacity) robot.charge = robot.capacity;

// Sensor sees Energy as Light until within 5*radius:
if (dist_to_energy < 5.0 * energy.GetRadius()) {
  // sensor target type becomes 'Energy'
} else {
  // sensor reads it as 'Light' (uses Light intensity, not Energy type)
}
```

6.3 Water Entity -- Charge Drain Source

Water entities represent physical water on the terrain. Any robot inside the water boundary suffers:

```
// Water drain formula:
robot.charge -= robot.capacity * 0.1 * dt;
// continuous drain at 10% max capacity per second
// Note: robot capacity, not current charge -- always a fixed high rate
```

Robots should use Fear-Water Braitenberg behavior to avoid this. A BV with an inverse-weighted motion will naturally fear water proportional to proximity.

Entity Type Hierarchy -- Class Relationships

IEntity (interface, core/) <-- EntityBase (sim/) <-- Robot
<-- BraitenbergVehicle
<-- Light <-- Energy
<-- Water

<-- Predator (wraps other entity via Decorator)

IMotion <-- DefaultMotion, BounceMotion, KeyboardMotion, AroundMotion, BackAndForthMotion
<-- CompositeMotion, ClosestSwitchMotion
<-- WeightedMotion (abstract) <-- PercentMotion, ChargeLevelMotion, InverseWeightedMotion
<-- DDMotionBase <-- Braitenberg <-- Love, Fear, Aggression, Explore

7 Interconnectedness -- How It All Fits Together

Understanding the full data flow from JSON to pixel is critical for the code review. Here is the end-to-end lifecycle of ONE simulation tick for a BraitenbergVehicle using InverseWeightedMotion with three Braitenberg children:

1. JSON Parsed SimulationModel::CreateScene() reads entities array

2. MotionFac

Critical Dependency Chain to Know for Code Review

- SimulationModel owns all IEntity* -> calls Update(dt) on each
- EntityBase owns IMotion* motion -> calls motion->Move(*this, dt)
- CompositeMotion / WeightedMotion own vector<IMotion*> children -> call child->Move()
- Braitenberg motions own EntitySensor* -> sensors query SimulationModel->GetEntities()
- SimulationModel is passed by const reference deep into the motion tree -- read-only
- ONLY EntityBase sub-classes (not IEntity) can be used with WeightedMotion (dynamic_cast needed)

8 MotionFactory -- The Central Wiring Hub

MotionFactory::Create() is a static factory method -- a single entry point that maps a JSON motion object to a concrete IMotion* instance. This is the glue between the data-driven scene format and the C++ class hierarchy. It is called recursively for composite/weighted motions so arbitrarily deep trees can be specified in JSON.

```
// src/sim/motion/MotionFactory.cpp  -- full dispatch table
IMotion* MotionFactory::Create(const json& motionJson, double W, double H,
                               const SimulationModel& model) {
  string type = motionJson.value("type", "default");
  double speed = motionJson.value("speed", 0.0);

  if (type == "default")        return new DefaultMotion();
  if (type == "bounce")         return new BounceMotion(speed, W, H);
  if (type == "back_and_forth") return new BackAndForthMotion(P1, P2, speed);
  if (type == "around")         return new AroundMotion(center, r, speed);
  if (type == "keyboard")       return new KeyboardMotion(up,right,down,left, speed, W, H);

  if (type == "composite") {
    auto* c = new CompositeMotion();
    for (auto& child : motionJson["children"])
      c->AddChild(Create(child, W, H, model));     // RECURSIVE!
    return c;
  }
  if (type == "closest-switch") { ... auto* cs = new ClosestSwitchMotion(model, types);
    for (auto& child ...) cs->AddChild(Create(child, W, H, model));  return cs; }

  if (type == "percent")        { ... return new PercentMotion(weights); }
  if (type == "charge-level")   { ... return new ChargeLevelMotion(thresholds); }
  if (type == "inverse-weighted") {
    auto* iw = new InverseWeightedMotion(model, entityTypes, searchRadius);
    for (auto& child ...) iw->AddChild(Create(child, W, H, model));  return iw;
  }
  return new DefaultMotion();  // fallback
}
```

SimulationModel::CreateEntity() calls MotionFactory::Create() ONCE per entity, passing the 'motion' sub-object from the entity's JSON. The entire motion tree (however deep) is built before the entity is constructed -- so the entity's constructor receives a fully wired IMotion*.

Design Pattern Summary Table

Pattern	Where Applied	Benefit
Strategy	IMotion + all concrete motions	Any entity uses any motion; swap at runtime via JSON
		Adapter

Top 5 Things to Know Cold for the Code Review

1. IMotion interface: single method Move(IEntity&, dt). EVERYTHING flows through this.
2. MotionFactory::Create(): recursive JSON dispatch. Know ALL type strings.
3. Composite vs Weighted: Composite runs ALL children sequentially; Weighted blends results.
4. EntitySensor uses typeid() at runtime -- NOT template/compile-time type checking.
5. Energy drain: charge -= (h_dist + 0.01) * (dt/50) * (radius/50). Water: 0.1*capacity*dt.